

CE 356 Project Weak Binary Semaphore

Team members: Tianbin Wang, Kehao Zhang

1. Background

In this CE356 project, our team focuses on the topic of **weak binary semaphores**. According to the project description, a weak binary semaphore is a binary semaphore with weak fairness. Unlike **Cache Coherence**, which is another topic listed in the course project suggestions, our project does not aim to model a complete cache coherence protocol. Instead, our emphasis is on understanding how to design correct mutual exclusion mechanisms for multiple threads or concurrent tasks when the underlying synchronization primitive provides only weak fairness, while preserving as much fairness as possible for waiting processes that attempt to enter the critical section.

More specifically, we study the following problem: multiple processes, threads, or tasks repeatedly compete for the same shared critical section, yet a weak binary semaphore by itself does not guarantee fair access among waiting processes. As a result, under the simplest mutual exclusion design, some waiting process may be overtaken repeatedly and may remain unable to enter the critical section for a long time. Under the restriction that only weak binary semaphores may be used, the central goal of this project is to construct an n -process mutual exclusion protocol and formally verify its correctness in TLA+.

2. Project Objective

The overall goal of this project is to investigate the limitations of weak binary semaphores by comparing a naive mutual exclusion design with an improved protocol, and by analyzing what benefits the more complex protocol actually provides. In particular, our study focuses on the following questions:

- What properties can the traditional single-**mutex** mutual exclusion design guarantee under weak semaphore semantics?
- Why can such a baseline satisfy mutual exclusion while still allowing individual starvation?
- How does the Udding algorithm use a more structured admission discipline to reduce or avoid unbounded overtaking of waiting processes?
- Does a stronger progress guarantee necessarily lead to a more complex TLA+ state space?

3. Planned Project Structure

To ensure that the project progresses steadily, we divide the work into several stages and intermediate goals.

3.1 Theoretical and Modeling Preparation

We first combine course materials with related references to build a foundational understanding of concurrent mutual exclusion problems and formal modeling methods. Our main references include:

- Chapter 4, especially **Peterson's Algorithm**, to understand the structure of classical mutual exclusion protocols;
- Chapter 5, especially the modeling of **Caching Memory**, to learn how a real concurrency problem can be abstracted into a formal state-machine model;

- operating systems and concurrency references, to strengthen our understanding of semaphores, mutual exclusion, deadlock, and starvation.

3.2 Baseline Model

After establishing the basic background, we first construct the most natural and traditional baseline: the single-**mutex** mutual exclusion design. It can be written as:

```
NCS;  
P(mutex);  
CS;  
V(mutex);
```

Here, **mutex** is a weak binary semaphore. The advantage of this baseline is that it is simple and intuitive, which makes it a useful point of comparison. It captures the basic idea of mutual exclusion very clearly, but it also exposes the key problem under weak semaphore semantics: although at most one process may enter the critical section at a time, some processes may still fail repeatedly in contention and thus experience individual starvation.

Therefore, the purpose of the baseline is not to provide the final answer, but to serve as a reference case that helps us explain why simple mutual exclusion does not imply fair access under weak fairness assumptions.

3.3 Udding Algorithm as the Main Protocol

Based on the baseline, we introduce the **Udding algorithm**, identified through literature review, as the core implementation target of this project. Unlike the direct contention of a single **mutex**, the Udding protocol organizes the progress of waiting processes through a more structured admission discipline. In particular, it uses multiple semaphores and auxiliary counters to distinguish processes that are already in the waiting pipeline from those that arrive later, thereby reducing the possibility that waiting processes are overtaken indefinitely.

The main value of the Udding protocol is that it does not merely preserve mutual exclusion. More importantly, it addresses the progress problem under weak semaphore assumptions, especially the risk of individual starvation. For this reason, it is a particularly suitable core protocol for this project.

4. Comparison Dimensions

To systematically present the differences between the baseline and the Udding protocol, we plan to compare them along the following three dimensions.

4.1 Mutual Exclusion

This is the most fundamental safety property and the basis of the project. We compare:

- whether the baseline satisfies mutual exclusion;
- whether Udding satisfies mutual exclusion.

The expected result is that both should satisfy mutual exclusion. In other words, at the safety level, both the baseline and Udding should ensure that at most one process is in the critical section at any time.

4.2 Starvation Risk

This is the most important and most valuable comparison dimension of the project. We focus on:

- why the baseline may lead to starvation under weak semaphore assumptions;
- how Udding avoids individual starvation through a more structured protocol design.

This part directly matches the difficulty highlighted by the project description: a single weak binary semaphore may be enough to protect the critical section, but it does not automatically guarantee fair access for waiting processes.

4.3 TLA+ State Complexity

Finally, we compare the complexity of the two models from the perspective of formal verification.

- the baseline TLA+ model has fewer states and a simpler structure, so it is easier to model and verify;
- the Udding TLA+ model contains more states, more actions, and a larger state space, so the verification cost is higher.

This dimension highlights an important fact: stronger behavioral guarantees often imply more complex formal models, and formal verification itself becomes correspondingly more challenging.

5. Prototype of baseline

5.1 The code of baseline

```
----- MODULE BaselineMutex -----
EXTENDS Naturals, FiniteSets

CONSTANT Proc

ASSUME Proc /= {}

VARIABLES pc, mutex

Vars == << pc, mutex >>

Init ==
  /\ pc = [i \in Proc |-> "ncs"]
  /\ mutex = 1

EnterTrying(i) ==
  /\ pc[i] = "ncs"
  /\ pc' = [pc EXCEPT ![i] = "trying"]
  /\ mutex' = mutex

AcquireMutex(i) ==
  /\ pc[i] = "trying"
  /\ mutex = 1
  /\ pc' = [pc EXCEPT ![i] = "cs"]
  /\ mutex' = 0
```

```

LeaveCS(i) ==
  /\ pc[i] = "cs"
  /\ pc' = [pc EXCEPT ![i] = "exit"]
  /\ mutex' = mutex

ReleaseMutex(i) ==
  /\ pc[i] = "exit"
  /\ mutex = 0
  /\ pc' = [pc EXCEPT ![i] = "ncs"]
  /\ mutex' = 1

ProcStep(i) ==
  EnterTrying(i)
  \/ AcquireMutex(i)
  \/ LeaveCS(i)
  \/ ReleaseMutex(i)

Next ==
  \E i \in Proc : ProcStep(i)

TypeOK ==
  /\ pc \in [Proc -> {"ncs", "trying", "cs", "exit"}]
  /\ mutex \in {0, 1}

InCS ==
  {i \in Proc : pc[i] = "cs"}

MutualExclusion ==
  Cardinality(InCS) <= 1

Spec ==
  Init /\ [][Next]_Vars

=====

```

In this Prototype, we define the baseline model described in Section 3.2. It formalizes the single-mutex semaphore procedure as a state-transition system, where each process repeatedly moves through NCS, trying, CS, and exit. The model is intentionally abstract: it focuses on the control protocol and allows TLC to explore all reachable interleavings and possible states, so that core safety properties such as mutual exclusion can be checked systematically.